MALWARE STATIC ANALYSIS REPORT

Linux ELF Post-Exploitation Agent — "PhantomRing"

SHA-256:

`2d7b1b2178f76c26893b2a56cbf9b36700235259e76b893d53817d5b66b634a5`

Sample Name	agent
Classification	Post-Exploitation / Remote Access Agent (Linux ELF)
Analysis Type	Static Analysis (no dynamic detonation performed)
Origin	Recovered from /var/tmp on a Linux host during SOC threat hunting
Analyst	RismaFareedh — Cybersecurity Analysis Coursework / Project
Report Date	September 10, 2026
Distribution	Educational / Internal Training Use Only

Table of Contents

1. Executive Summary 3
2. Sample Overview 3
3. Analysis Environment & Methodology 4
4. Detailed Static Analysis Findings 5
    4.1 Network Communication & C2 Infrastructure 5
    4.2 Command & Control Capability Set 6
    4.3 Defense Evasion 7
        4.3.1 io_uring-Based Syscall Evasion 7
        4.3.2 eBPF Security-Tool Fingerprinting and Termination 7
        4.3.3 ftrace / Kernel Tracing Subsystem Tampering 8
    4.4 Host Reconnaissance 8
        4.4.1 Logged-In User Enumeration 9
    4.5 Privilege Escalation Support 9
    4.6 Anti-Forensics: Remote Self-Destruction 9
        4.6.1 Self-Location via /proc/self/exe 9
        4.6.2 The Real Trigger Token: “sdestruct”, Not “selfdestruct” 9
5. MITRE ATT&CK Mapping 10
6. Indicators of Compromise (IOCs) 11
7. Risk Assessment 11
8. Detection & Mitigation Recommendations 12
9. Conclusion 13
Appendix A: Static Analysis Command Reference 14

1. Executive Summary

During a routine threat-hunting sweep, the SOC team recovered an unstripped 64-bit Linux ELF binary named agent from /var/tmp on a production host. The file was actively attempting outbound network connections and exhibited naming and placement patterns consistent with a post-exploitation implant staged by an intruder who already had code-execution on the box. This report documents a full static analysis of the sample — no dynamic detonation or sandboxing was required, as every capability, configuration value, and indicator below was recovered directly from the binary's symbol table, strings, and disassembly.

The sample is a fully featured Linux remote-access agent (RAT) that beacons out to a hardcoded command-and-control (C2) host, accepts eleven distinct operator commands, and bundles a purpose-built defense-evasion module. Most notably, the agent performs all of its file and network I/O through the Linux io_uring asynchronous I/O interface rather than calling connect(), read(), write(), or open() directly. This routes its activity around the classical syscall entry points that most EDR and eBPF-based monitoring tools hook, allowing the agent to communicate and manipulate the filesystem largely “off the radar” of userspace-syscall-hooking security products — an evasion pattern that has become increasingly common in modern Linux malware.

Beyond C2 and evasion, the agent supports host reconnaissance (logged-in users, running processes, active network connections), a privilege-escalation helper that enumerates SUID binaries, and an active counter-detection routine that fingerprints and kills processes with loaded eBPF security tooling and disables the kernel ftrace subsystem. A dedicated self-destruct command allows the operator to remotely and irrecoverably delete the implant from disk, complicating incident response and forensic recovery.

VirusTotal community scanning flagged the sample as malicious by 23 of 63 signature-based engines at time of submission — a moderate detection rate that underlines the value of the static, capability-driven analysis in this report over signature matching alone. Key indicators of compromise, a MITRE ATT&CK mapping of observed behaviors, and concrete detection/mitigation guidance are provided in Sections 5–8.

2. Sample Overview

Basic identification and hashing is always the first step of analysis: it lets the sample be pivoted against threat-intel platforms and gives a stable reference point that survives any renaming.

Attribute	Value
File Name	agent
File Size	30.46 KB (31,192 bytes)
File Type	ELF 64-bit LSB pie executable, x86-64, dynamically linked
Interpreter	/lib64/ld-linux-x86-64.so.2
Linked Libraries	glibc 2.7 (dynamic), liburing (io_uring)
Compiler	GCC (Ubuntu 11.4.0-1ubuntu1~22.04.2) 11.4.0
Symbols	Not stripped — original function names intact (cmd_*, process_cmd, etc.)
MD5	6d9a243b881984aa25fc0e770dc0634c
SHA-1	6580406b3487c7bc32ae8024add3715131f53d24
SHA-256	2d7b1b2178f76c26893b2a56cbf9b36700235259e76b893d53817d5b66b634a5

Attribute	Value
BuildID (SHA-1)	1f617f2ea259a7ec724d7bbc01627982dc2f0495
SSDEEP	768:MXVVXtldVNF91tldVNF91tldVNF91tldVNoAYwlg4QoAa0dRgmAzw1MI:Mt5Mw1M
Vhash	4c1eb79f20c52085430c7634f43e790e
TLSH	T1FEE2A31BB291DF38E4D4F2301BDBD6E0A62078F06736315F275546B72AB33984B78A46
VirusTotal Detection	23 / 63 security vendors (community score)
Discovery Location	/var/tmp on the affected Linux host

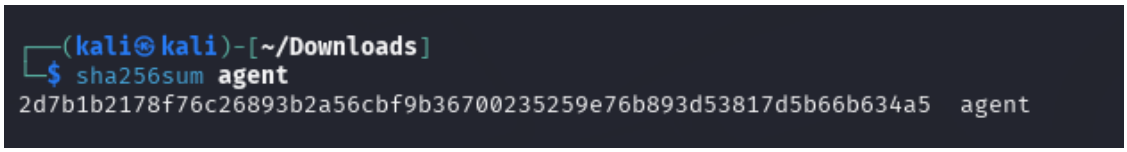


Figure 1. SHA-256 hash computed with sha256sum.

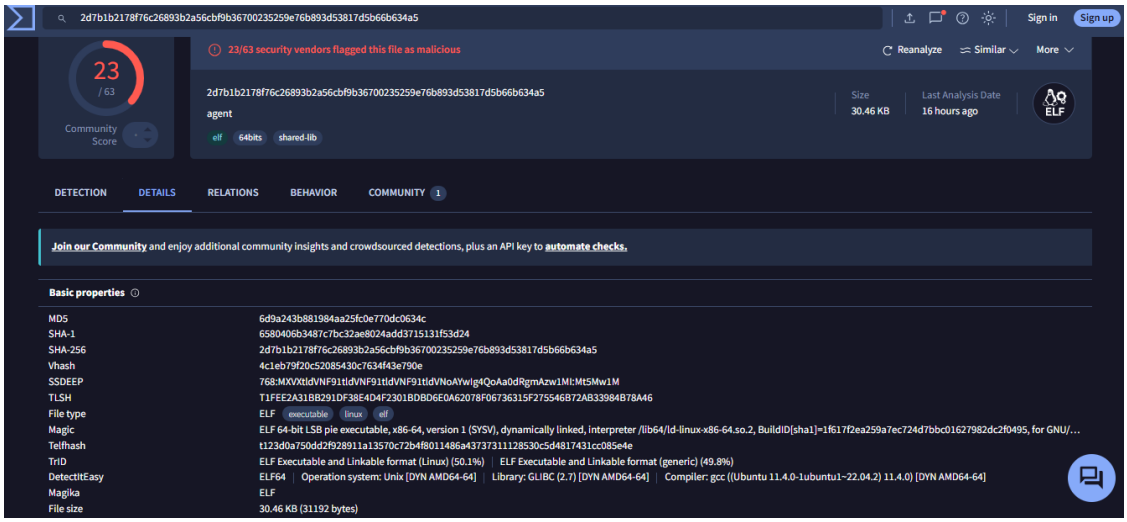


Figure 2. VirusTotal detail view — 23/63 vendors flag the sample as malicious; hashes, TLSH, SSDEEP, and compiler metadata confirmed.

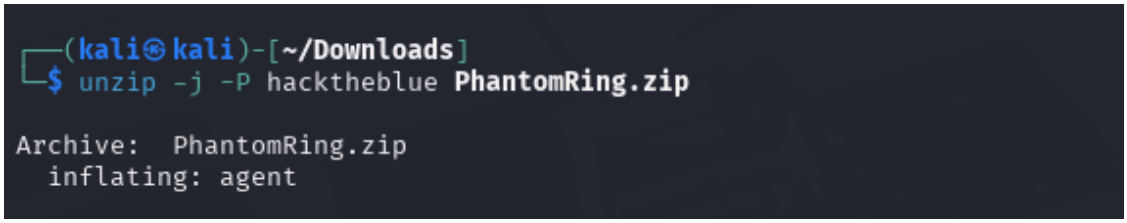


Figure 3. Extraction of the sample from the password-protected evidence archive (chain-of-custody step).

3. Analysis Environment & Methodology

All analysis was performed statically on a Kali Linux workstation, without executing the sample. Two tools did the bulk of the work:

- strings -a agent — extracts printable ASCII strings from every section of the binary (paths, error messages, command tokens, IP addresses).

- objdump -d -M intel agent / objdump -s -j .rodata agent — disassembles executable code in Intel syntax and dumps raw section contents, used whenever a value (port numbers, timer values, byte-exact command tokens) is not recoverable from strings alone.

A recurring theme in this analysis is that strings only surfaces printable text: numeric constants such as port numbers and sleep durations are compiled directly into instruction operands and are invisible to a strings pass. These required reading the disassembly around the relevant library call (e.g. htons(), sleep()) and converting the hexadecimal operand to decimal. Because the binary is not stripped, the original function names (cmd_users, cmd_killbpf, process_cmd, etc.) remained in the symbol table, which made capability enumeration and cross-referencing dramatically faster than a blind reverse-engineering effort would have required.

4. Detailed Static Analysis Findings

4.1 Network Communication & C2 Infrastructure

The agent links against liburing and performs its outbound connection through io_uring_prep_connect() rather than a direct connect() call (see Section 4.3.1 for the evasion implication). The destination IP is stored as a plain ASCII dotted-quad string, while the destination port is a raw integer operand loaded immediately before the call to htons() and only visible in disassembly.

Parameter	Value
C2 IP Address	192.168.56.1
C2 Port	TCP/4445
Reconnect Backoff	120 seconds (sleep(0x78)) after a failed connection or a dropped socket
Transport Mechanism	io_uring asynchronous I/O (io_uring_prep_connect, io_uring_prep_send, io_uring_prep_recv)



Figure 4. C2 IP address recovered via a dotted-quad regex over strings output.



Figure 5. Disassembly showing 0x115d loaded into edi immediately before call htons — 0x115d = 4445 decimal, confirming the C2 port.

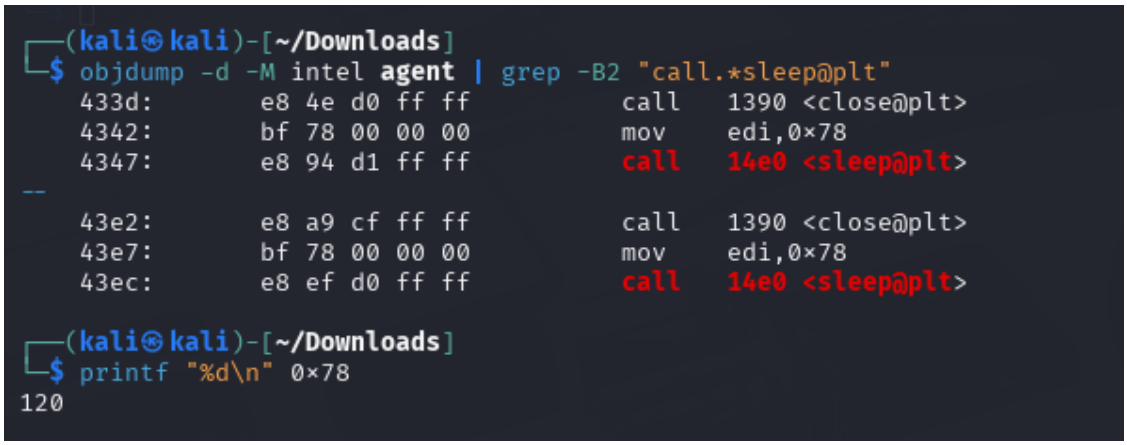


Figure 6. Both reconnect-path sleep() calls load 0x78 (120 decimal) into edi, confirming the 120-second backoff.

The 192.168.56.1 address falls in a private/lab (VirtualBox host-only) range, consistent with this sample being a training/CTF artifact rather than a live internet-facing C2; in a genuine incident this field is the primary network IOC to block and hunt on.

4.2 Command & Control Capability Set

Because the binary retains its symbol table, every command handler is directly enumerable as a cmd_* function, giving an exact and complete capability count rather than an estimate:



Figure 7. Full symbol-table enumeration of cmd_* handlers — 11 distinct commands.

Any operator input that does not match one of the eleven tokens below falls through to a catch-all branch that returns "[*] 404 Command not found [*]" and is not counted as a supported command.

Command	Handler Addr.	Category	Function
users	0x1e49	Reconnaissance	Parses /var/run/utmp to enumerate currently logged-in users.
ss	0x20a3	Reconnaissance	Reads /proc/net/tcp and formats active TCP connections (Local/Remote Address, State, UID) — an in-agent reimplementation of ss/netstat.
get	0x23da	Collection	Retrieves a file from the victim host for exfiltration to the operator (inferred from symbol name and paired recv functionality).

Command	Handler Addr.	Category	Function
recv	0x2635	Ingress Tool Transfer	Receives a file pushed by the operator and writes it to a specified remote path and size (usage string: “recv <remote_path> <size>”).
me	0x29b0	Reconnaissance	Reports the agent's own PID, controlling TTY, and command name (via /proc/self/comm).
ps	0x2a86	Reconnaissance	Lists running processes (PID / CMD) by walking /proc.
kick	0x2cfc	Impact / Control	Terminates a specified PID or session at the operator's request.
privesc	0x32d8	Privilege Escalation	Scans /usr/bin for SUID binaries as local privilege-escalation candidates.
sdestruct	0x3546 (cmd_selfdestruct)	Anti-Forensics	Resolves its own path via /proc/self/exe and unlinks (deletes) the binary from disk.
exit	0x36ea	Session Control	Terminates the current agent session/connection.
killbpf	0x37a2	Defense Evasion	Disables kernel ftrace controls and kills processes that have an eBPF map loaded (security tooling).

4.3 Defense Evasion

4.3.1 io_uring-Based Syscall Evasion

Rather than calling connect(), open(), read(), write(), or close() directly, the agent submits every I/O operation as a request on an io_uring submission queue and lets the kernel complete it asynchronously. Because many EDR products and eBPF-based monitors instrument the traditional syscall entry points (via ptrace, seccomp-bpf hooks, or LD_PRELOAD interposition), work routed through io_uring can bypass exactly the observation points those tools rely on — the syscalls the security product expects to see never appear where it is watching.

```
┌──(kali㉿kali)-[~/Downloads]
└─$ strings -a agent | grep -i "io_uring"
io_uring_queue_init
io_uring_queue_exit
io_uring_submit
__io_uring_get_cqe
io_uring_get_sqe
io_uring_queue_init
io_uring_wait_cqe: %s
io_uring_wait_cqe
io_uring_cq_advance
io_uring_cqe_seen
io_uring_prep_rw
io_uring_prep_connect
io_uring_prep_openat
io_uring_prep_close
io_uring_prep_read
io_uring_prep_write
io_uring_prep_statx
io_uring_prep_send
io_uring_prep_recv
io_uring_prep_unlinkat
io_uring_wait_cqe_nr
io_uring_wait_cqe
io_uring_submit@LIBURING_2.0
io_uring_queue_exit@LIBURING_2.0
io_uring_queue_init@LIBURING_2.0
__io_uring_get_cqe@LIBURING_2.0
io_uring_get_sqe@LIBURING_2.0
```

Figure 8. Full set of liburing symbols imported by the agent, covering connect, open, read, write, send, recv, statx, and unlink operations — essentially the entire I/O surface of the malware is routed through io_uring.

This is a growing technique in Linux malware and rootkit development; defenders should not assume syscall-hooking EDR provides complete visibility on hosts where io_uring is unrestricted (see Section 8 for mitigation guidance).

4.3.2 eBPF Security-Tool Fingerprinting and Termination

The killbpf command walks /proc/[pid]/maps for every running process and searches for the substring anon_inode:bpf-map — the pseudo-file name the kernel exposes in a process's memory map whenever that process has an eBPF map loaded. Because modern Linux security and observability tools (e.g. Falco, Tetragon, Cilium-based monitors) rely on eBPF, this lets the agent fingerprint exactly which running PIDs belong to security tooling and terminate them.

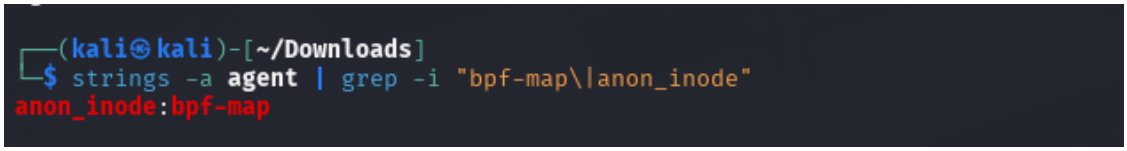


Figure 9. The anon_inode:bpf-map fingerprint string used to identify eBPF-instrumented processes.

4.3.3 ftrace / Kernel Tracing Subsystem Tampering

The same killbpf routine also writes to three ftrace control files under /sys/kernel/debug/tracing to blind kernel-level tracing. Disassembly of cmd_killbpf shows the three file-path strings are loaded into an array in a fixed order — tracing_on, then set_event, then current_tracer — and the disable loop processes index 0 first, meaning the master tracing_on switch is disabled before the other two control files are touched.

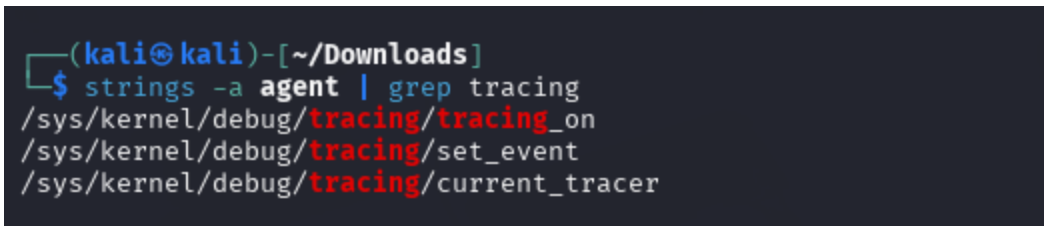


Figure 10. The three ftrace control-file paths referenced by the binary.

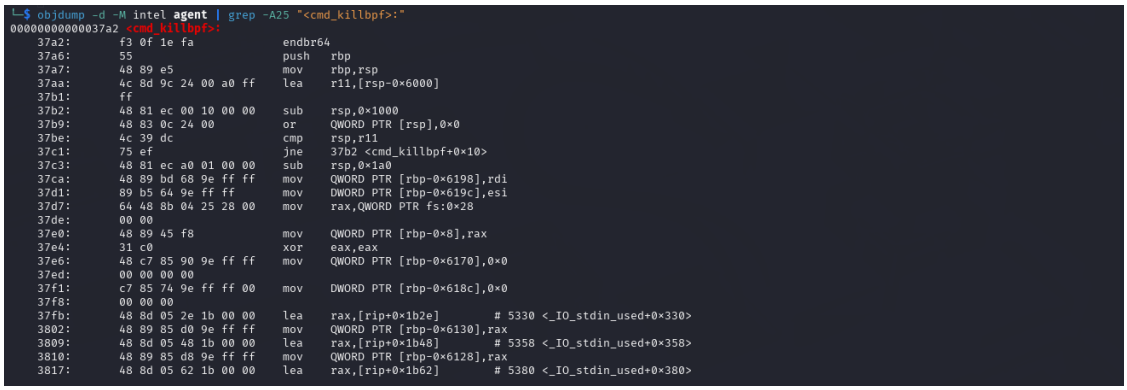


Figure 11. Disassembly of cmd_killbpf showing the three tracing-file addresses (0x5330, 0x5358, 0x5380) loaded in sequence.

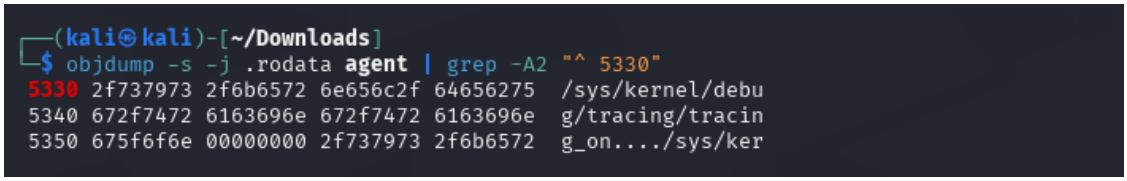


Figure 12. Raw .rodata bytes confirming /sys/kernel/debug/tracing/tracing_on is the first string in the array (offset 0x5330).

Order	Tracing File Disabled
1st	/sys/kernel/debug/tracing/tracing_on

Order	Tracing File Disabled
2nd	/sys/kernel/debug/tracing/set_event
3rd	/sys/kernel/debug/tracing/current_tracer

4.4 Host Reconnaissance

Two commands drive host reconnaissance beyond the general ps/ss enumeration already covered in Section 4.2: user enumeration and privilege-escalation surveying.

4.4.1 Logged-In User Enumeration

The users command opens and parses /var/run/utmp directly, the same binary database read by the standard who and w utilities, rather than shelling out to an external binary — this keeps the agent's activity from spawning a child process that a process-monitoring EDR rule would flag.

```
┌──(kali㉿kali)-[~/Downloads]
└─$ strings -a agent | grep -i utmp
/var/run/utmp
Error reading /var/run/utmp
```

Figure 13. /var/run/utmp path and its matching error string, confirming direct file parsing.

4.5 Privilege Escalation Support

The privesc command enumerates /usr/bin, building a full path for every entry (format string /usr/bin/%s) and checking each file's permission bits for the SUID flag — a standard first move in Linux local-privilege-escalation reconnaissance, since a misconfigured SUID binary can often be abused to gain root.

```
┌──(kali㉿kali)-[~/Downloads]
└─$ strings -a agent | grep -B2 -A2 "SUID"
/usr/bin
Failed to open /usr/bin
Potential SUID binaries:
/usr/bin/%s
Agent will self-destruct
```

Figure 14. SUID-scan strings, including the target directory /usr/bin and the "Potential SUID binaries:" result banner.

4.6 Anti-Forensics: Remote Self-Destruction

The agent supports remote, irreversible self-deletion — a significant complication for incident response, since a successfully triggered self-destruct removes the primary artifact from disk before it can be imaged or collected.

4.6.1 Self-Location via /proc/self/exe

Before deleting itself, the agent reads the /proc/self/exe symlink, which the kernel always keeps pointing at the real on-disk path of the running process's own executable. This lets the malware find its own file path at runtime regardless of the working directory it was launched from, so it can target the correct file for deletion.

```
┌──(kali㉿kali)-[~/Downloads]
└─$ strings -a agent | grep "proc/self"
/proc/self/exe
```

Figure 15. The /proc/self/exe string used for self-location prior to deletion.

4.6.2 The Real Trigger Token: "sdestruct", Not "selfdestruct"

This is the most easily misread finding in the sample. The handler function is literally named cmd_selfdestruct, and the binary even prints a human-readable status message, "Agent will self-destruct." It is tempting to assume the operator-facing command is "selfdestruct" — but the literal token compared by strcmp() is shorter than the function name implies.

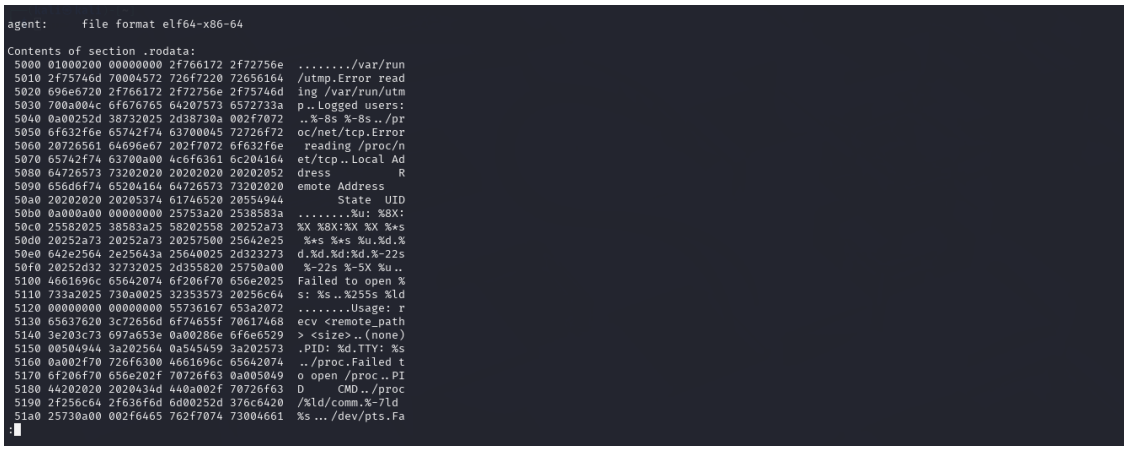


Figure 16. Raw .rodata dump of the command-token table. Reading across the 0x54c0→0x54d0 line boundary spells s-d-e-s-t-r-u-c-t, not "selfdestruct" — there is no "self" prefix in the actual bytes.

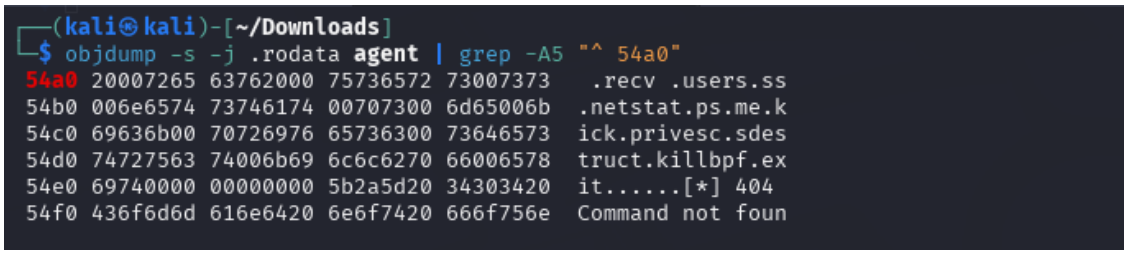


Figure 17. Disassembly of the command dispatcher confirming strcmp() compares operator input directly against the address holding "sdestruct" (0x54cc) before branching into cmd_selfdestruct.

This discrepancy between the descriptive internal function name and the literal wire-format command token is a useful general lesson: string and symbol names describe intent, but only the literal bytes compared at runtime are authoritative for building detections (e.g. a YARA rule or network signature keyed on the command word must use "sdestruct", not "selfdestruct", to actually match).

5. MITRE ATT&CK Mapping

Mapping the recovered capabilities onto MITRE ATT&CK for Enterprise (Linux platform) situates this sample against a common reference framework for detection engineering and threat reporting.

Tactic	Technique	Evidence in Sample
Command and Control	T1071 — Application Layer Protocol	Beacons to a hardcoded C2 host over a raw TCP session.
Command and Control	T1571 — Non-Standard Port	Connects out on TCP/4445, a non-standard, non-registered port.

Tactic	Technique	Evidence in Sample
Defense Evasion	T1562.001 — Impair Defenses: Disable or Modify Tools	killbpf terminates processes that have an eBPF security tool's map loaded.
Defense Evasion	T1562.006 — Impair Defenses: Indicator Blocking	killbpf disables tracing_on, set_event, and current_tracer under /sys/kernel/debug/tracing, blinding kernel-level tracing.
Defense Evasion	T1070.004 — Indicator Removal: File Deletion	sdestruct command reads /proc/self/exe and unlinks its own binary.
Defense Evasion	(Emerging) io_uring Syscall-Hook Evasion	All file/network I/O is routed through io_uring, bypassing classic syscall-hooking EDR/eBPF instrumentation. Not yet a dedicated ATT&CK sub-technique at time of writing; closely related to T1562.001 in effect.
Discovery	T1033 — System Owner/User Discovery	users command parses /var/run/utmp for logged-in accounts.
Discovery	T1057 — Process Discovery	ps command enumerates running processes via /proc.
Discovery	T1049 — System Network Connections Discovery	ss command parses /proc/net/tcp for active connections.
Discovery	T1082 — System Information Discovery	me command reports the agent's own PID/TTY/command context.
Privilege Escalation	T1548.001 — Abuse Elevation Control Mechanism: Setuid/Setgid	privesc enumerates /usr/bin for SUID binaries.
Collection	T1005 — Data from Local System	get command retrieves files from the victim host.
Command and Control	T1105 — Ingress Tool Transfer	recv command receives operator-pushed files onto the victim host.
Impact / Session Control	T1489-adjacent (Kick/Exit)	kick and exit provide the operator remote session/process termination control.

6. Indicators of Compromise (IOCs)

Indicator	Value
SHA-256	2d7b1b2178f76c26893b2a56cbf9b36700235259e76b893d53817d5b66b634a5
SHA-1	6580406b3487c7bc32ae8024add3715131f53d24
MD5	6d9a243b881984aa25fc0e770dc0634c
C2 IP Address	192.168.56.1
C2 Port	TCP/4445
Filename Observed	agent
Drop Location	/var/tmp
Command Tokens	users, ss, get, recv, me, ps, kick, privesc, sdestruct, exit, killbpf
eBPF Fingerprint String	anon_inode:bpf-map
ftrace Files Tampered	/sys/kernel/debug/tracing/{tracing_on, set_event, current_tracer}
Recon File Read	/var/run/utmp

Indicator	Value
SUID Scan Directory	/usr/bin
Self-Location Path	/proc/self/exe

The command tokens, tracing-file paths, and the eBPF fingerprint string are particularly high-value for host-based detection: their appearance in a process's memory, argv, or open-file list is far more specific than the hash alone, which will not survive recompilation.

7. Risk Assessment

Factor	Assessment
Confidentiality Impact	High — arbitrary file retrieval (get), process/user/network enumeration, and full remote command execution context.
Integrity Impact	Medium–High — the recv command can push and write arbitrary files to the host; privesc supports follow-on local privilege escalation.
Availability Impact	Low–Medium — kick can terminate processes; no destructive payload beyond self-deletion was identified.
Detectability (pre-mitigation)	Low — io_uring routing and ftrace/eBPF-tool termination are specifically designed to defeat common Linux EDR visibility.
Anti-Forensic Capability	High — the sdestruct command can remove the only on-disk artifact of the intrusion at operator command.
Attribution / Sophistication	Moderate-to-high tradecraft: non-trivial choice of io_uring for evasion and active counter-eBPF logic go beyond a generic off-the-shelf RAT.

8. Detection & Mitigation Recommendations

8.1 Network-Based Detections

- Block and alert on outbound connections to 192.168.56.1:4445; more generally, alert on outbound connections to uncommon high ports (4444/4445-class) from server workloads that should not be initiating them.
- Baseline expected outbound destinations per host/segment and alert on new, unlisted destinations rather than relying on this single IP, which is trivially changed by the threat actor.

8.2 Host- and Kernel-Level Detections

- Deploy or update eBPF-based monitoring (Falco, Tetragon, or equivalent) with io_uring-aware probes (hooking io_uring_enter/io_uring_submit and related tracepoints); many older eBPF rule sets only watch classic syscalls and will miss io_uring-routed activity entirely.
- Where io_uring is not operationally required, restrict it via the io_uring_disabled sysctl (kernel 6.6+) or a seccomp profile that blocks the io_uring_setup/io_uring_enter/io_uring_register syscalls on servers that do not need it.
- Alert on writes to /sys/kernel/debug/tracing/tracing_on, set_event, or current_tracer from unexpected processes — legitimate tracing tools are typically well-known and infrequent.

- Alert on processes reading /proc/[pid]/maps across many other PIDs in rapid succession (a signature of the eBPF-tool-hunting behavior in killbpf), and consider running security tooling with names/paths that are harder to fingerprint generically.
- Monitor for a process reading its own /proc/self/exe followed shortly by an unlink() on that same path — a strong signature for in-place self-deletion regardless of the exact command word used.

8.3 File & Privilege Hygiene

- Run periodic SUID/SGID audits on /usr/bin and other common binary directories, and file-integrity-monitor for newly introduced SUID bits.
- Treat any unexplained executable in /var/tmp (or other world-writable directories) as suspicious by default; consider mounting such directories noexec where operationally feasible.

8.4 Threat Hunting

- Hunt for the IOCs in Section 6 across EDR telemetry, proxy/firewall logs, and endpoint file systems.
- Build a YARA rule keyed on the literal command-token bytes (users, ss, get, recv, me, ps, kick, privesc, sdestruct, exit, killbpf) and the anon_inode:bpf-map fingerprint string — not on the more “obvious” but incorrect selfdestruct string — to catch this family or close variants.

9. Conclusion

Static analysis alone — hashing, strings, and disassembly with objdump — was sufficient to fully characterize this sample without ever executing it: its C2 destination and timing, its complete eleven-command capability set, and a deliberate, multi-layered defense-evasion strategy built around io_uring, eBPF-tool termination, and ftrace tampering. The fact that the binary was left unstripped considerably accelerated this analysis; a stripped or packed variant of the same family would require heavier reverse engineering to recover the same level of detail.

The most operationally significant finding is the sample's evasion design: routing I/O through io_uring specifically sidesteps the syscall-hooking assumption that much of today's Linux EDR tooling is built on, while the killbpf command actively hunts and kills the very eBPF-based tools that would otherwise catch it. Any Linux security program should treat io_uring-capable, eBPF-aware evasion as a live and growing threat category, not a theoretical one, and should validate that its monitoring stack has genuine visibility into io_uring-routed activity rather than assuming syscall coverage is complete.

This report, its IOC list, and its MITRE ATT&CK mapping are provided as a static-analysis-driven detection and hunting baseline; a full incident response would additionally require host timeline analysis, log correlation, and controlled dynamic analysis in an isolated sandbox to confirm real-world network behavior and any capabilities not evident from static inspection alone.

Appendix A: Static Analysis Command Reference

Commands used throughout this analysis, for reproducibility:

```
sha256sum agent
strings -a agent | grep -E "^[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}$"
objdump -d -M intel agent | grep -B5 "call.*htons"
objdump -d -M intel agent | grep -B2 "call.*sleep@plt"
objdump -d agent | grep "^0000.*<cmd_"
strings -a agent | grep -i "io_uring"
strings -a agent | grep -i utmp
strings -a agent | grep -B2 -A2 "SUID"
strings -a agent | grep -i "bpf-map\|anon_inode"
strings -a agent | grep tracing
objdump -d -M intel agent | grep -A25 "<cmd_killbpf>:"
objdump -s -j .rodata agent | grep -A2 "^ 5330"
strings -a agent | grep "proc/self"
objdump -s -j .rodata agent | grep -A5 "^ 54a0"
objdump -d -M intel agent | grep -B4 "call.*strcmp@plt"
```